

Empty Product for certain Views

- Document number: P2540R1
- Date: 2022-02-06
- Author: Steve Downey <sdowney2@bloomberg.net>, <sdowney@gmail.com>
- Audience: SG9, LEWG

Abstract: This paper argues that the Cartesian product of no ranges should be a single empty tuple, which is the identity element for Cartesian products. Other product-like views, however, should not automatically have their identity be the result, and in particular for `zip`, as it would introduce unsound inconsistencies.

1. Motivation

A natural extension of a product of two things is to a product of (n) things, that is from $(P = A \times B)$ to $(P = \prod_{i=1}^n a_i = a_1 \cdots a_n)$, where the $(\prod)$ symbol stands for a repeated product, the same way that $(\sum)$ stands for a repeated sum. For $(n=1)$, the expansion immediately suggests that $(P=a_1)$, but the $(n=0)$ case requires more care. From the general rule that $(P_n a_{n+1} = P_{n+1})$, we have that $(P_0 a_1 = a_1)$, so (P_0) is the identity for the product. This convention simplifies induction arguments and sometimes improves consistency with other well defined operations. For example, having $(0^0 = 1)$, just as $(n^0 = 1)$ for all non-zero numbers, greatly simplifies Taylor series notation.

Also note that we are assuming up to isomorphism for types, and in particular that $((a, b, c))$ is isomorphic to $((a, b), c)$, and $((a, (b, c)))$, and further that the unit type $(())$ does not contribute, so that $(((), a) \equiv (a) \equiv (a, ()))$. That is there exists a simple and mechanical bijective mapping, one-to-one and onto, between the types.

Making the empty product the identity element also puts `fold` on a sounder footing. We don't have to supply an identity element because the base case gives it to us automatically.

The Cartesian product can also be viewed as the union of all relations between sets. Any subset of the Cartesian product is a relation among the sets. For any number of sets, there are always the trivial relations of $(\top)$ (every combination is in the relation) and $(\bot)$ (the relation is empty). With zero sets, those are the only two relations, since there is no input variation on which the value might depend. Accordingly, the Cartesian product of zero sets must have exactly one element (which is the empty tuple $(())$) so as to have exactly the empty set and the whole set as relations.

The most general definition of product comes from Category Theory, of course, where it is well studied. And an important result is that for a Category, such as sets, there is one universal operation that is the product. This should make us suspicious of extending the empty product $\equiv$ identity rule to other operations.

In particular, `zip` has the property that it is the inner join of the indexed sets, and is the main diagonal of the Cartesian product. However, the identity element for `zip` is `repeat(tuple<>)`, the infinite range of repeated empty tuples. If we allowed `zip` of an empty range of ranges to be its identity element, we would be introducing an inconsistency into the system, where two different formulations of notionally the same thing produces different answers. That would be bad.

2. Proposal

Specify that the Cartesian product of an empty range of ranges is a range of one element, which is the empty tuple, `std::tuple<>`. The type `std::tuple<>` is a monostate type, consisting of one element. This design should **not** be extended to `zip`. If it were to be defined, the `zip` of an empty range of ranges should be the diagonal of the

Cartesian product, but this is not actually useful, since that is annihilating for zip. It should be left undefined, as most operations on empty ranges are.

3. Wording

Wording is relative to p2374r3

3.1. Overview [range.cartesian.overview]

`cartesian_product_view` presents a view with a value type that represents the cartesian product of the adapted ranges.

The name `views::cartesian_product` denotes a customization point object. Given a pack of subexpressions `Es...`, the expression `views::cartesian_product(Es...)` is expression-equivalent to

- ~~`*decay-copy*(views::empty<tuple<>>)`~~ `views::single(tuple())` if `Es` is an empty pack,
- otherwise, `cartesian_product_view<views::all_t<decltype((Es))>...>(Es...)`.

4. References

Reflector Discussion: [isocpp-lib-ext] zip and cartesian_product base case <https://lists.isocpp.org/lib-ext/2022/01/22023.php>

Twitter: <https://twitter.com/sdowney/status/1482469504248598532> and ff

Empty product - Wikipedia: https://en.wikipedia.org/wiki/Empty_product

[P2374R3] Sy Brand, Michał Dominiak. 2021-12-13. `views::cartesian_product` <https://wg21.link/p2374r3>